

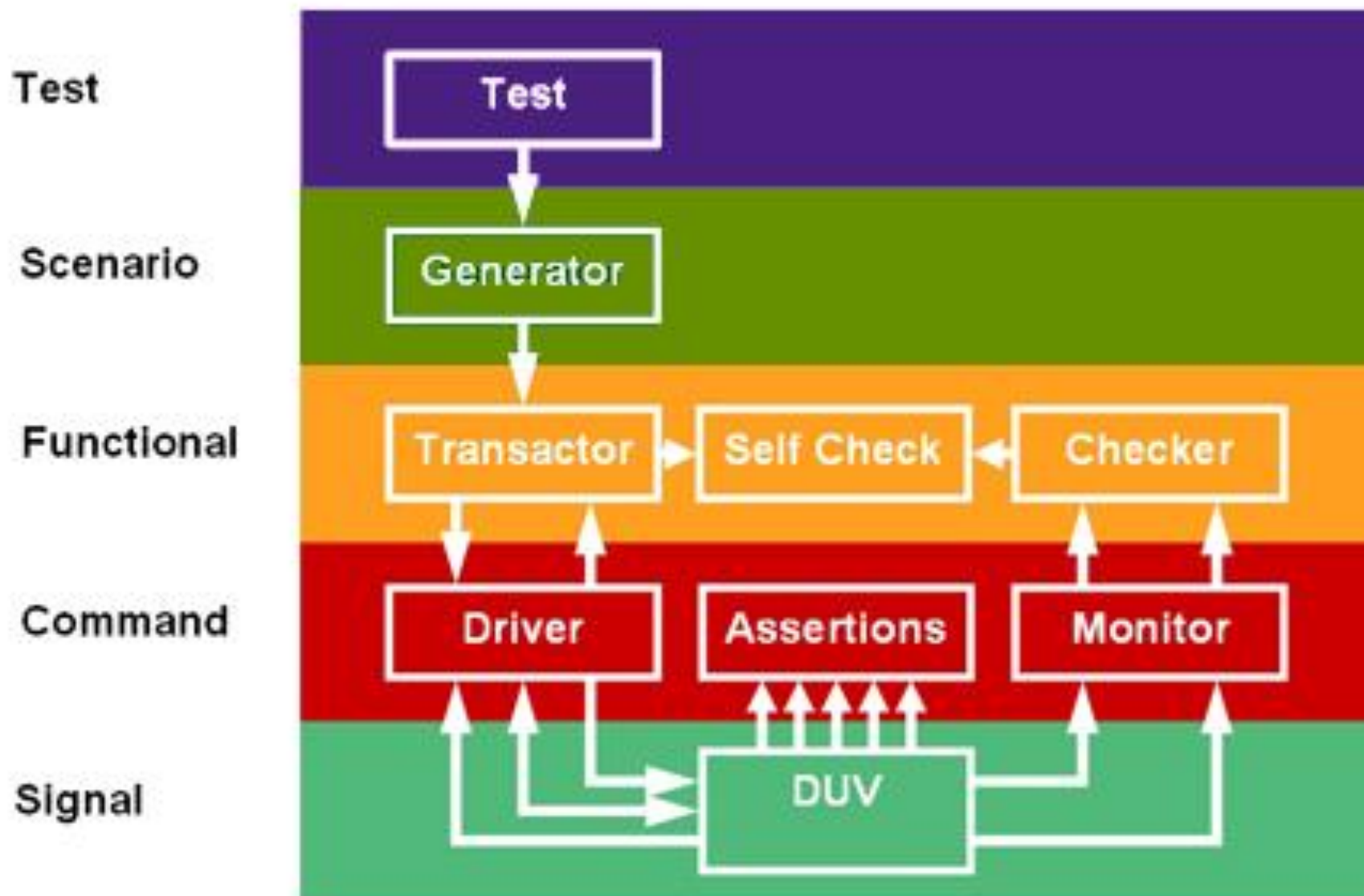
בדיקת מערכות ספרתיות ואימות תכנון

פרק 3 : סימולציה

חלק 3

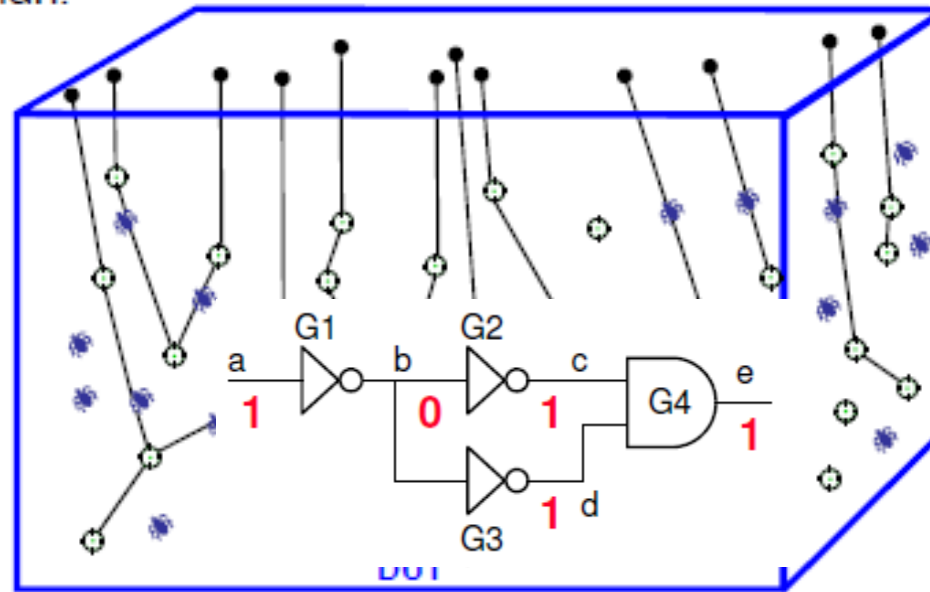


סביבות אימות



סביבת אימות: בדיקות מכונות

Verification Plan:

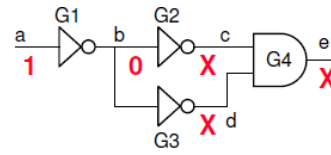
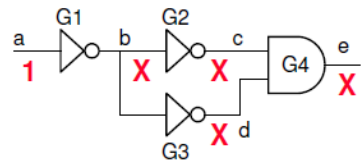
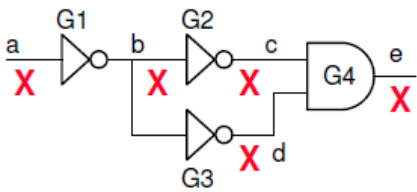


Redo if design changes

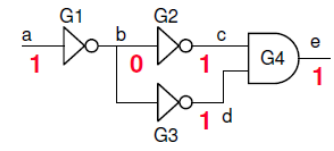
Automation Significant manual effort to write all the tests

Automation Work required to verify each goal was reached

Completeness Poor coverage of non-goal scenarios
... especially the cases that you didn't "think of"

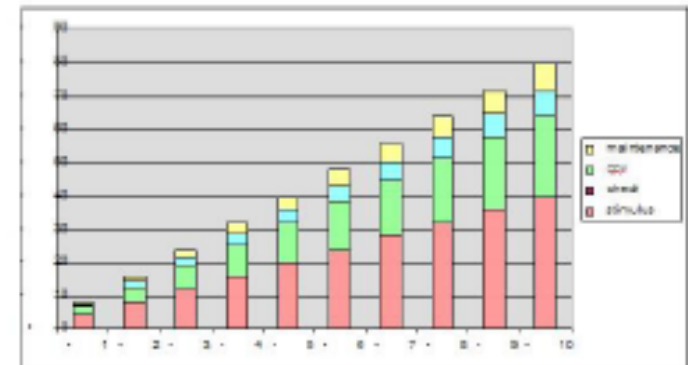
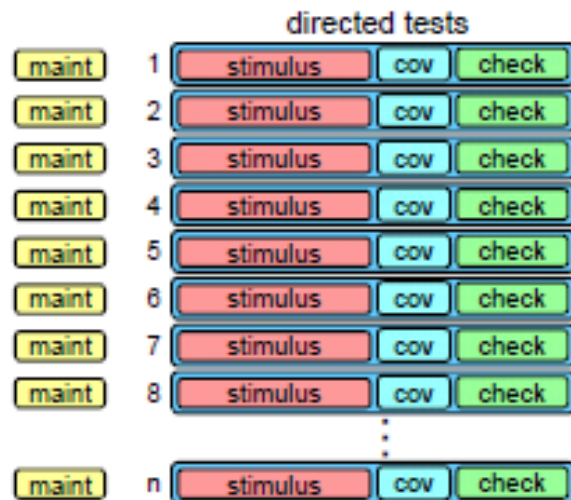


...

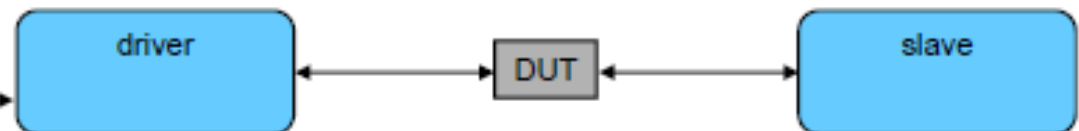


סביבת אימות: בדיקות מכוונות

- The more tests you have the more effort is required to develop and maintain them

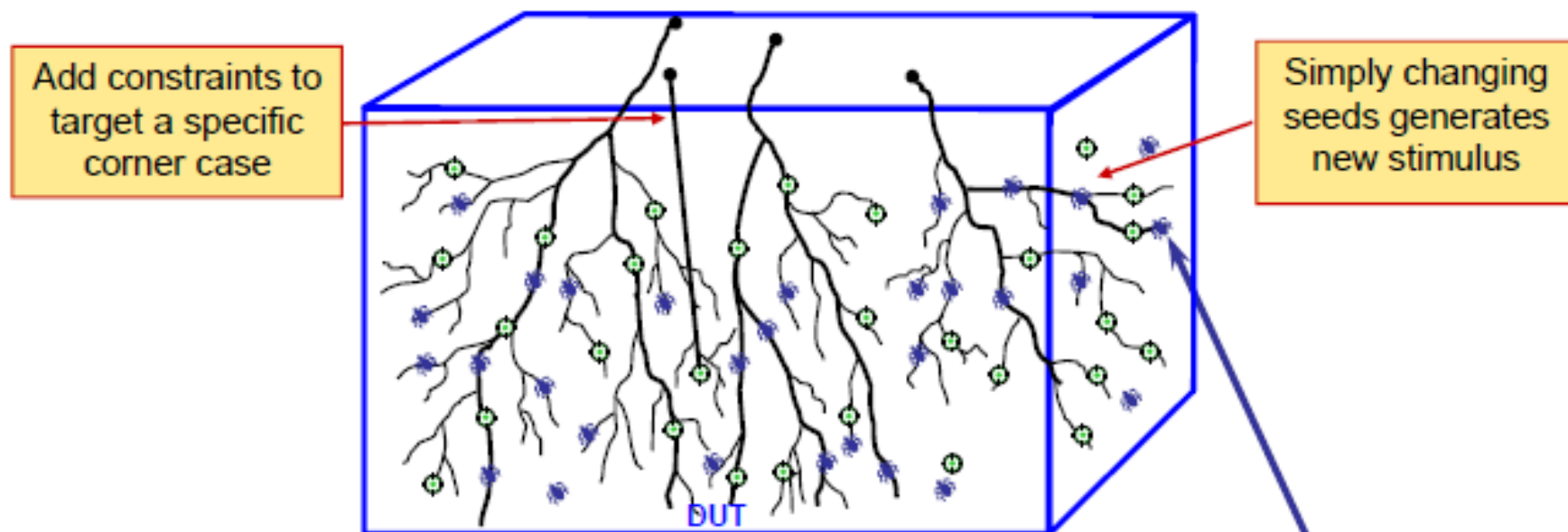


Directed test approach



סביבת אימות: בדיקות מבוססת כיסוי

Focuses on reaching goal areas (*versus execution of test lists*):



Constrained-random stimulus **generation** explores goal areas (& beyond)

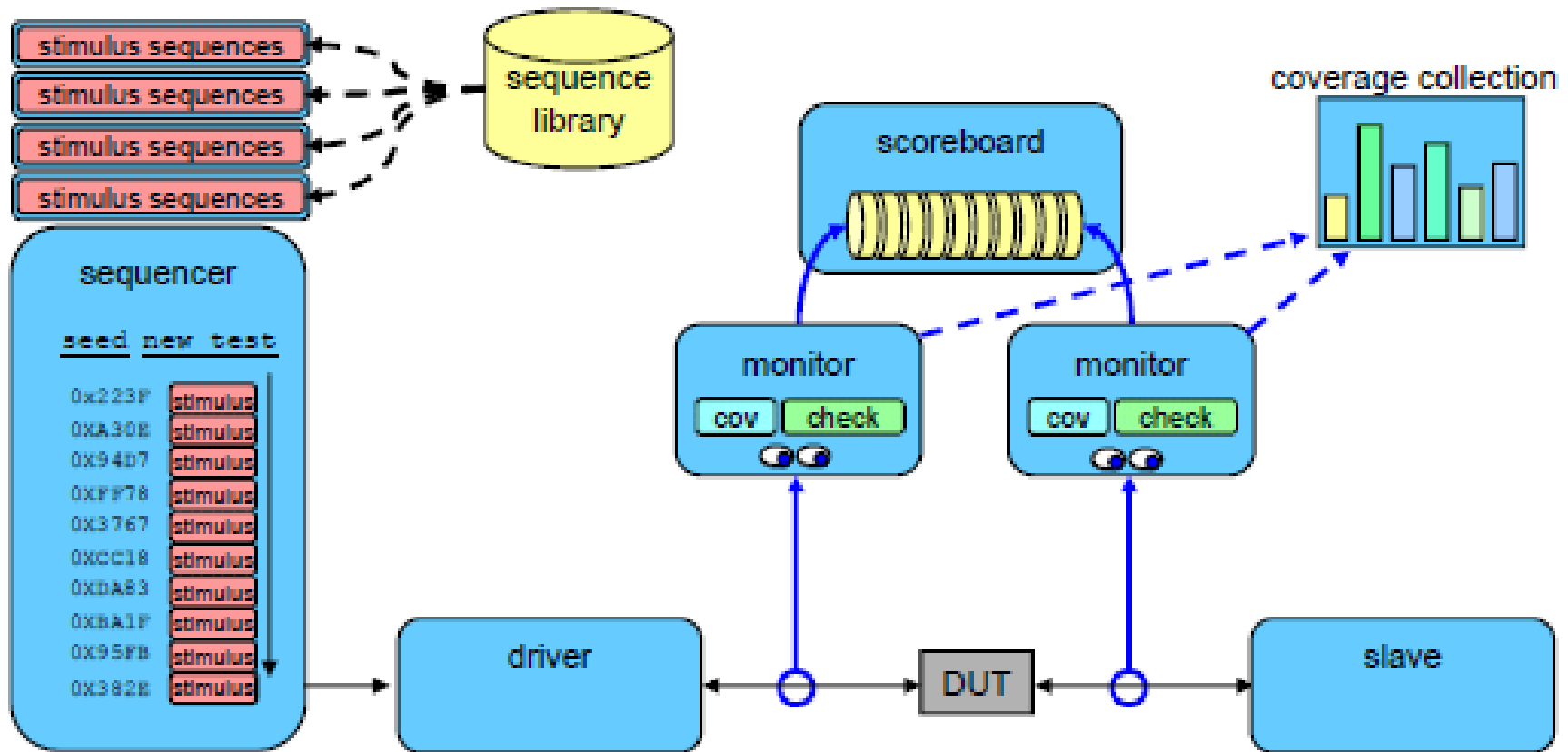
Coverage shows which **goals** have been exercised and which need attention

(Self-Checking ensures proper DUT response.)

Even for non-goal states!

Automation – Constrained-random stimulus accelerates hitting coverage goals and exposing bugs. Coverage and checking results indicate effectiveness of each simulation, which enables scaling many parallel runs.

סביבת אימות: בדיקות מבוססת כיסוי

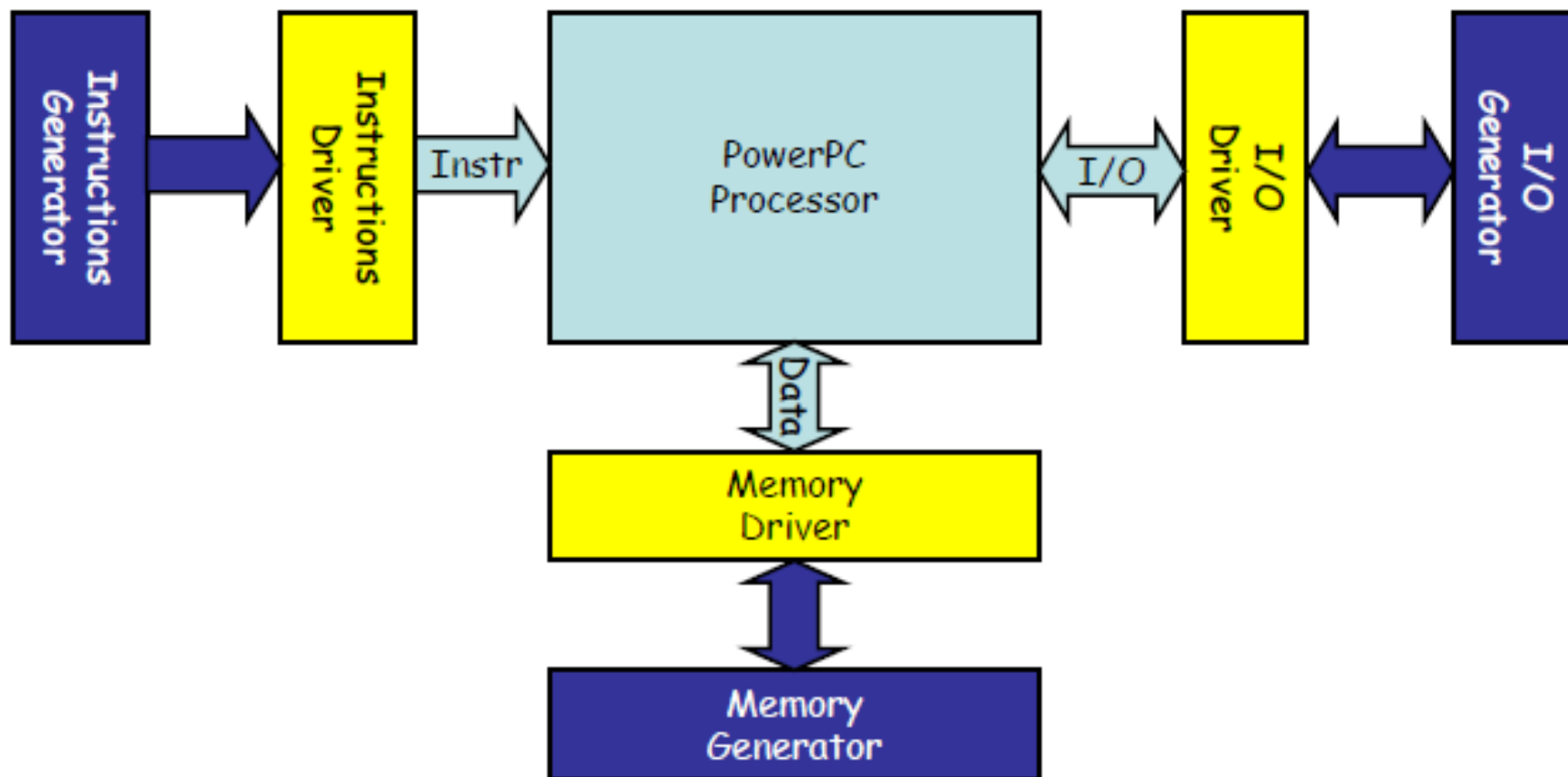


יצירת גירויים -- Stimulus Generation

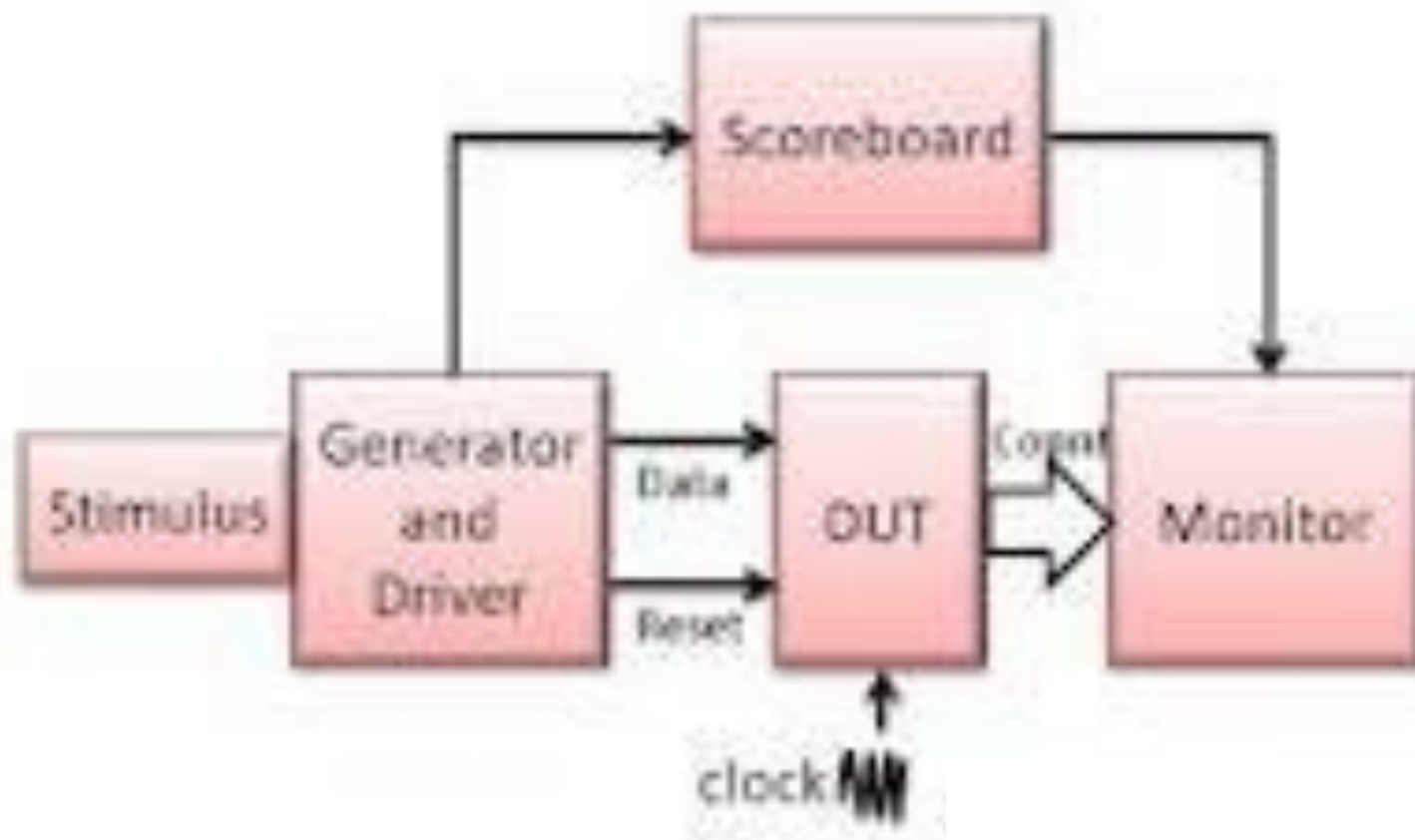
מטרות של יצירת גירויים:

- להשיג את כל האופציות בתרחישים של תכנית הבדיקות של האימות
- להראות את כל הבעיות שלא חשבנו עליהם בתוכנית אימות
- לספק מידע על תכנון לעזור לשחזר ולהבין את הבעיות

דוגמא: הסביבת אימות מעבד



יצירת גירויים

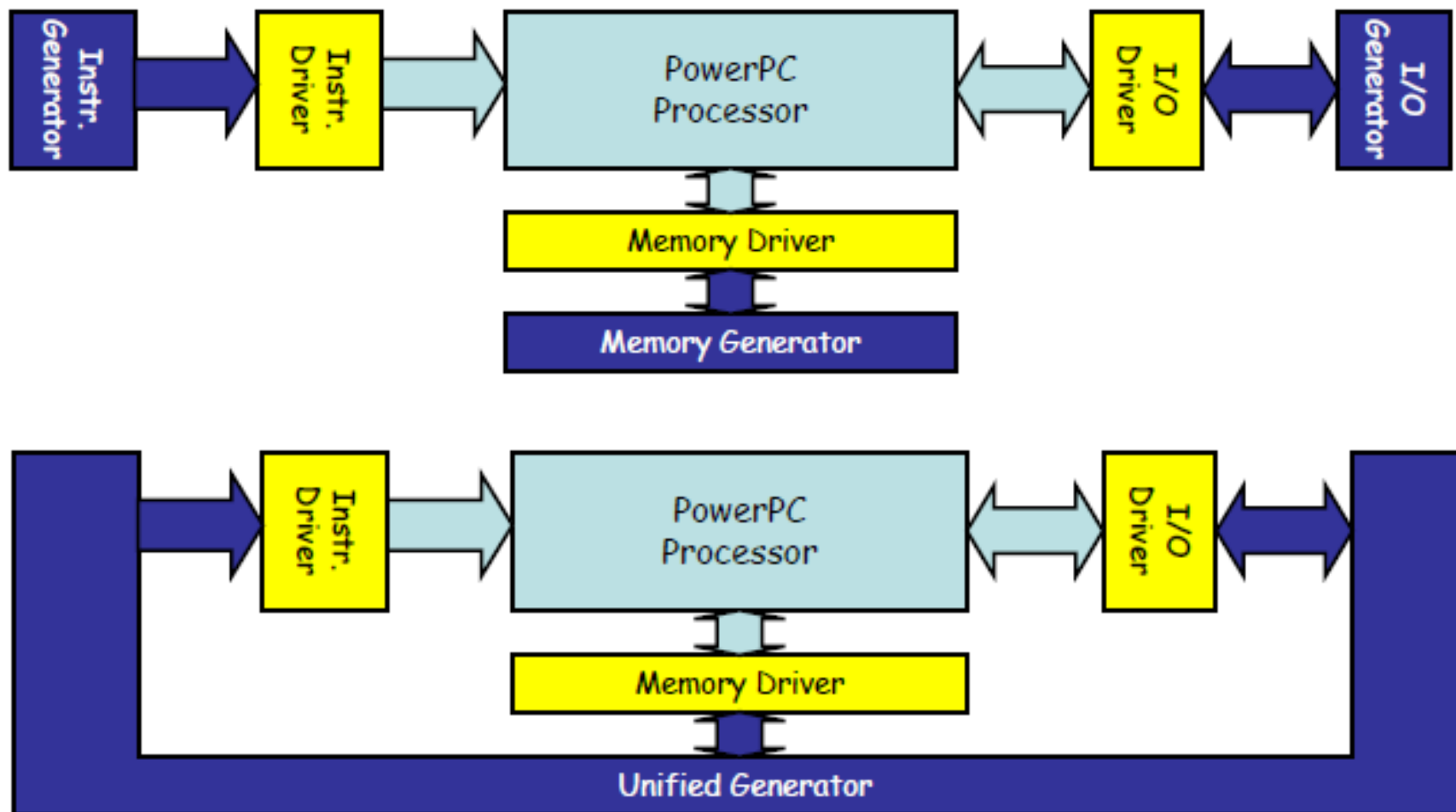


נושאים יצירת גירויים

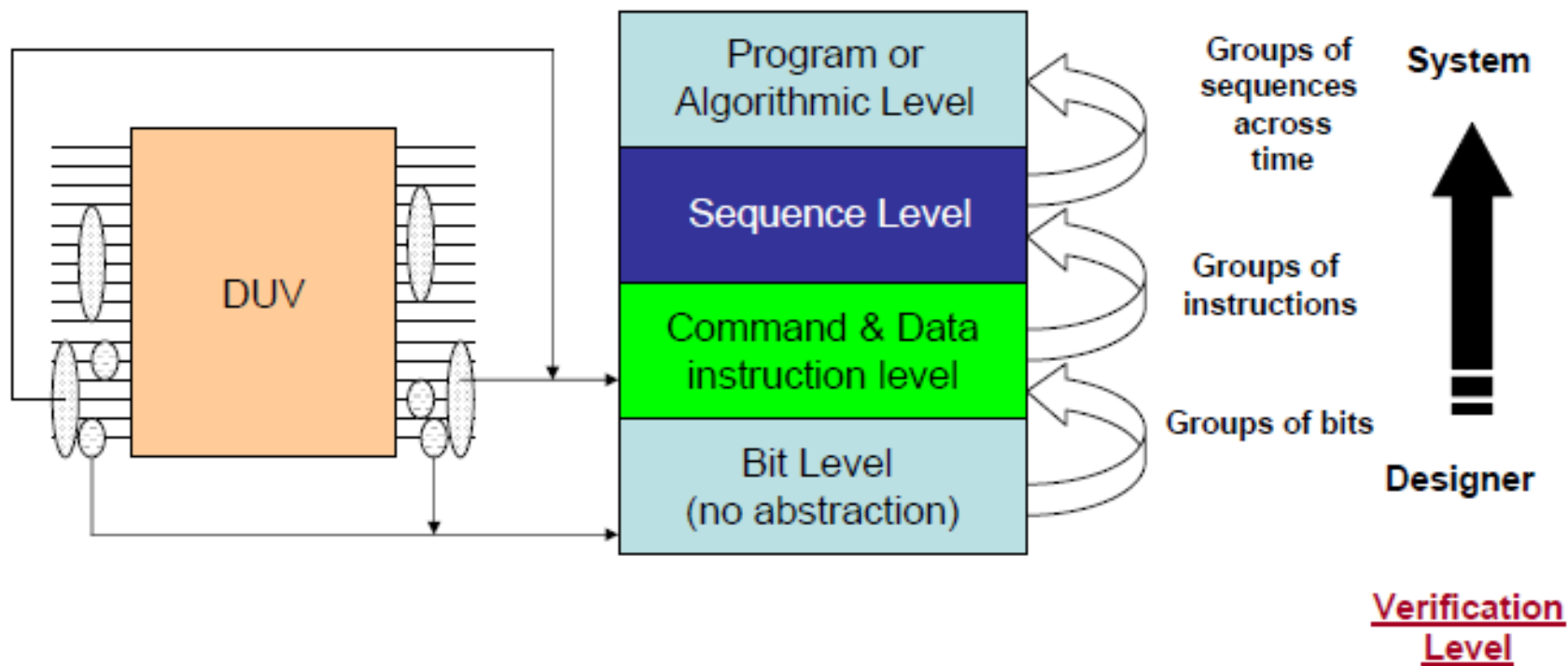


- בכמה גנרטורים להשתמש?
- באיזה רמה של הפשטה?
- Online vs. Offline יצירה
- Dynamic vs. Static יצירה
- אורך הבדיקות וטסטים
- אקראיות

בכמה גנרטורים להשתמש?



באיזה רמה של הפשטה?



Online Vs Offline

Online generation (on-the-fly) ➤

- הגנרטור הוא חלק מסביבת אימות
- הגנרטור יכול להשתמש במידע על מצב של הסביבה של DUV עבור שיפור איכות הגירויים
- הגנרטור חייב ליצור גירויים בסדר
- סיבוכיות מוגבלת

Online Vs Offline

Offline generation (pre-run) ➤

- ניתן להפריד את הגנרטור מהסימולציה
- אפשר להשתמש באלגוריתמים מורכבים יותר לייצור
- צריך לשלב את הגנרטור בסביבת האימות
- לא ניתן להשתמש במידע מן DUV והסביבה
- קשה ליצור את המגיבים לגירויים

Dynamic Vs. Static

➤ כל גנרטור סטטי אינו מודע על מצבו של DUV והסביבת האימות

- החלטות יצירה מבוססות על המצב הפנים של הגנרטור
- הגדרה פחות מוגבלת : הגנרטור מודע מה ומתי מותר ליצור

➤ כל גנרטור דינמי מודע על מצבו של DUV וסביבת האימות, ה גנרטור מייצר גירויים על סמך מידע זה

- הגנרטור יכול להגיב למצבים מעניינים של DUV

אורך הטסט

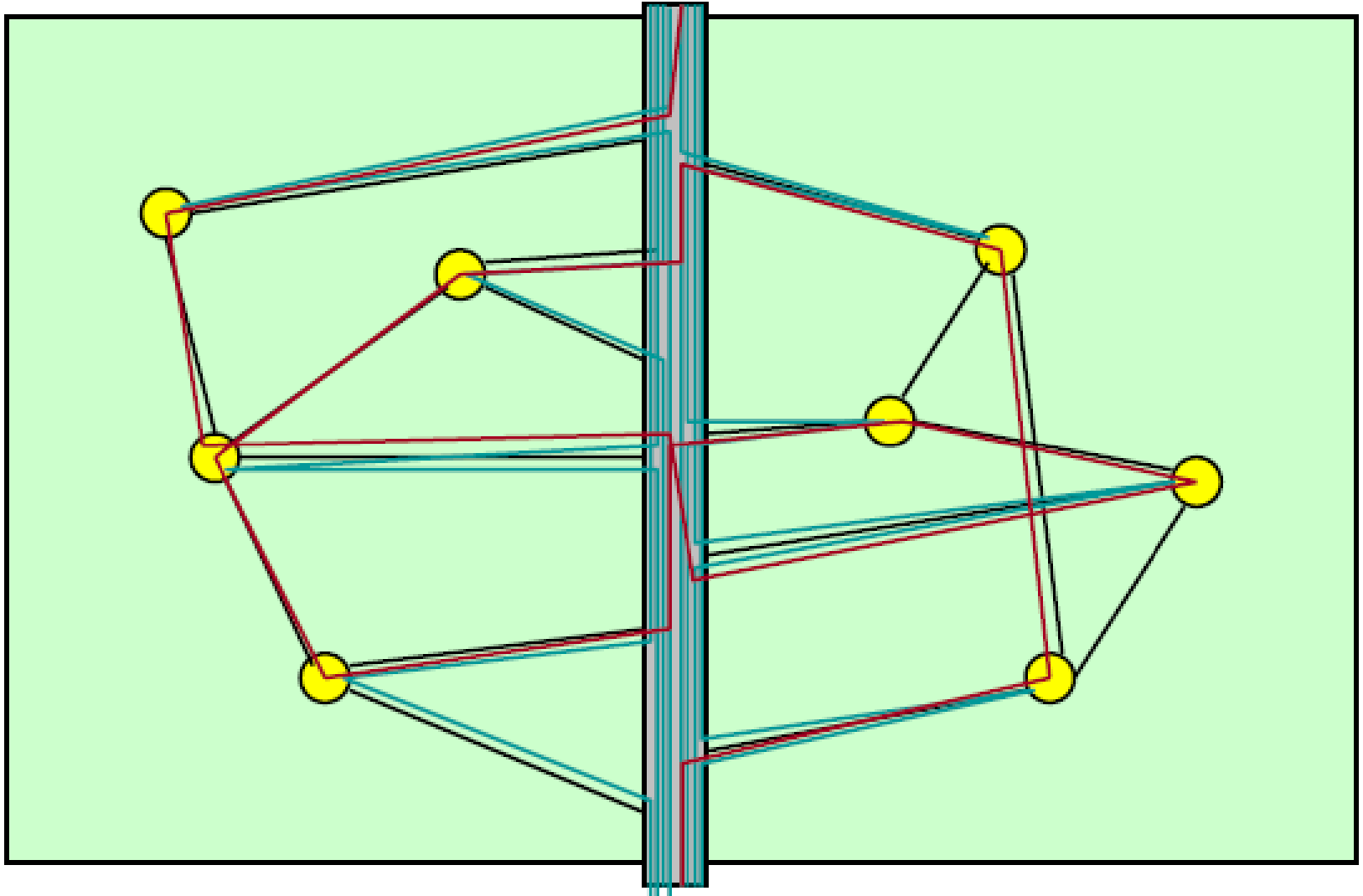
➤ בדיקות קצרות

- הבדיקות הקצרות ביותר שיכולות את הדרישות של תוכנית האימות
- עבור זוגות פקודות נדרשות בדיקות עם רק שתי פקודות

➤ בדיקות ארוכות

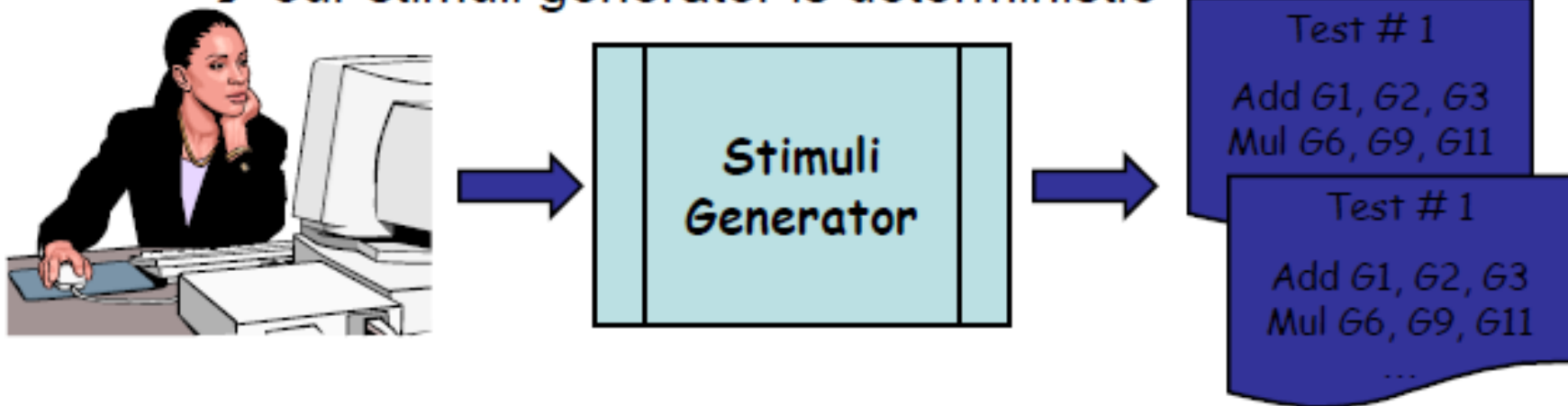
- שילוב דרישות רבות בבדיקה יחידה
- שילוב בדיקה עם סדרות התחלתיות

Short Vs. Long



אקראיות - מוטיבציה

- The same test appeared
→ our stimuli generator is deterministic



למה דטרמיניסטי?

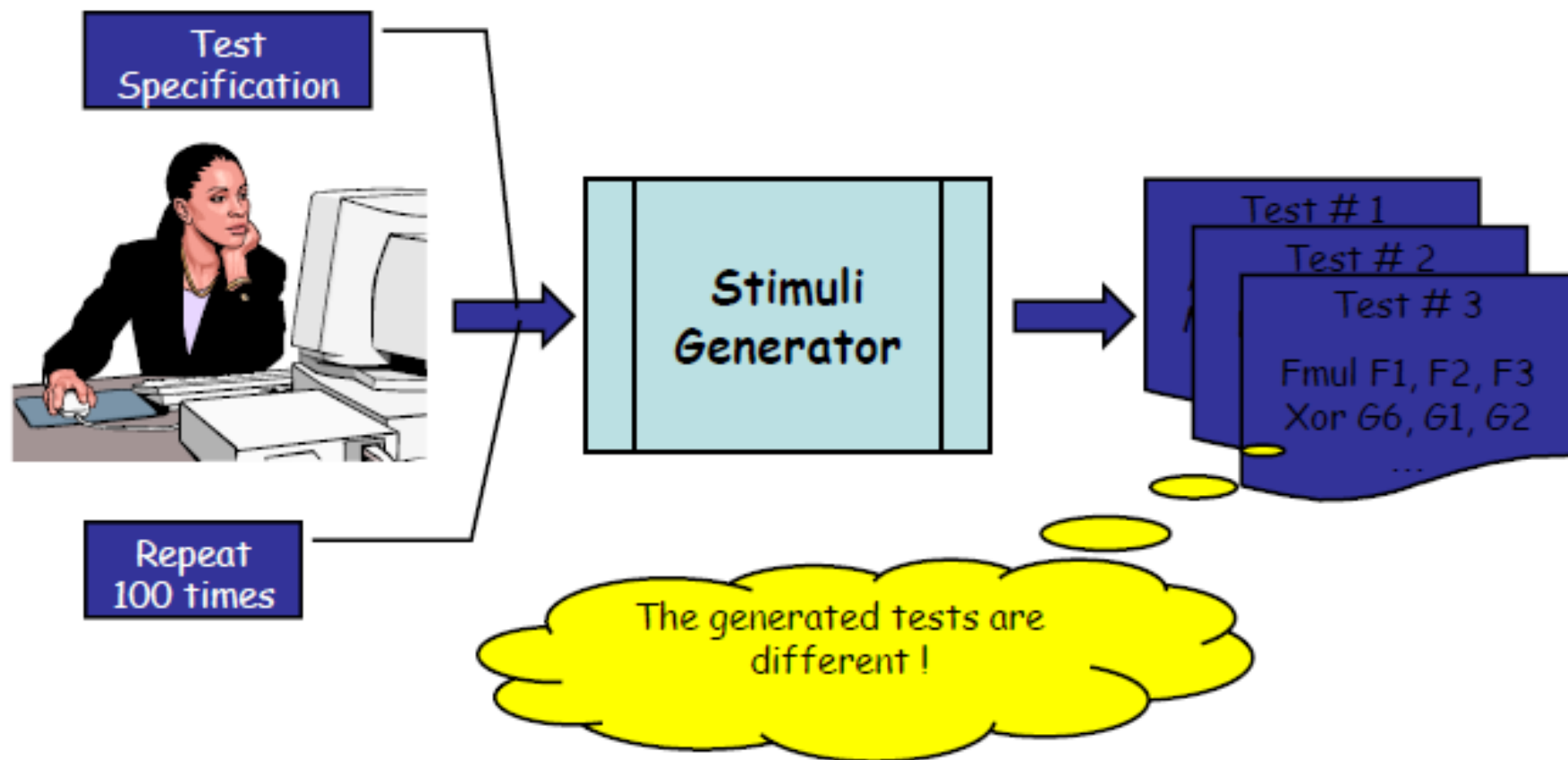
➤ ניתן לשימוש לפני שסביבה אקראית מוכנה

- הרבה יותר קל ליצור דרייוור שקורא בדיקות דטרמיניסטית ו מזריק אותן ל-DUV
- חבילות בדיקה שנרכשו קודם לכן
- האיכות הידועה
- מונע זמן יצירה ארוך מאוד

למה לא דטרמיניסטי?

- בדיקה הנתונה ניתן להשתמש רק פעם אחת
- יכולות מוגבלות בשימוש חוזר במפרט הבדיקות
- מתודולוגיית אימות מודרנית עובדים על תחנות העבודה רבות לסימולציה של טסטים רבים

יצירת בגירויים אקראיים



יצירה אקראית

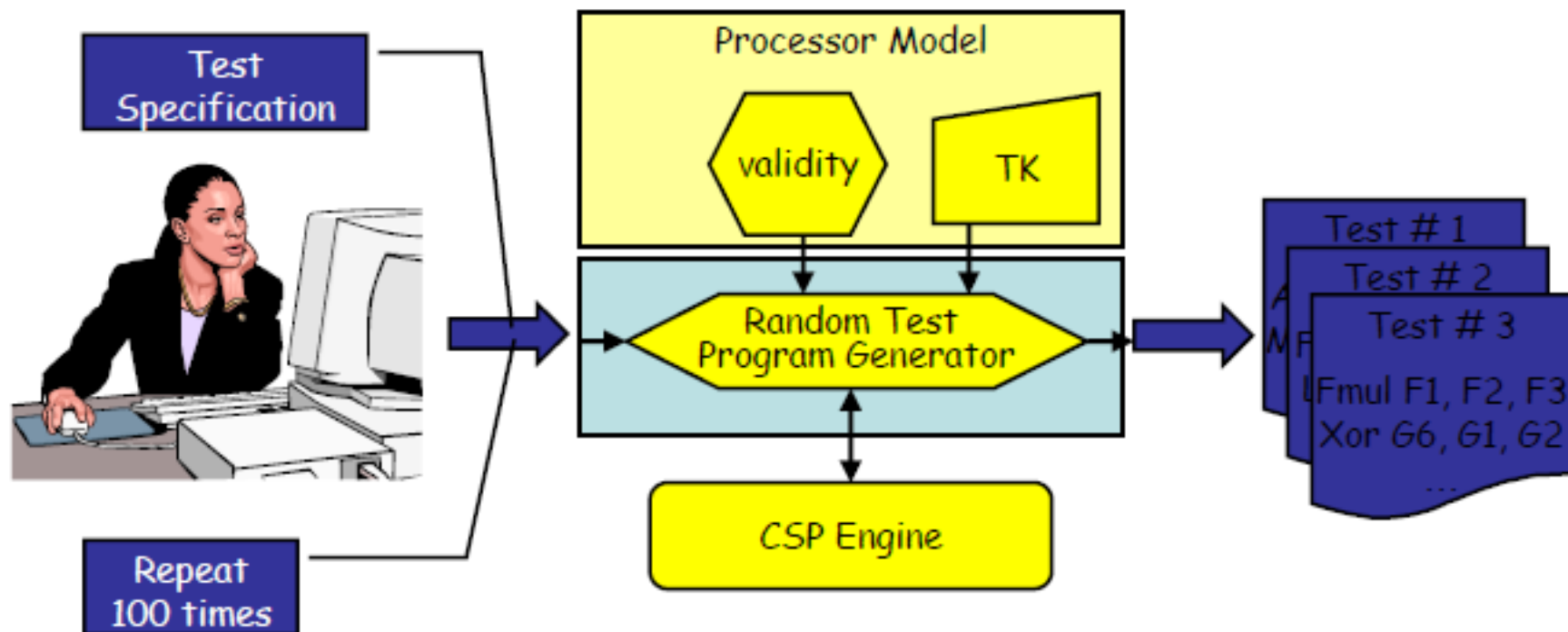
➤ יצירת אקראית טהורה

- הגנרטור מייצר סדרות אקראיות של '0' ו'1' שהן ארוזות כפקודות

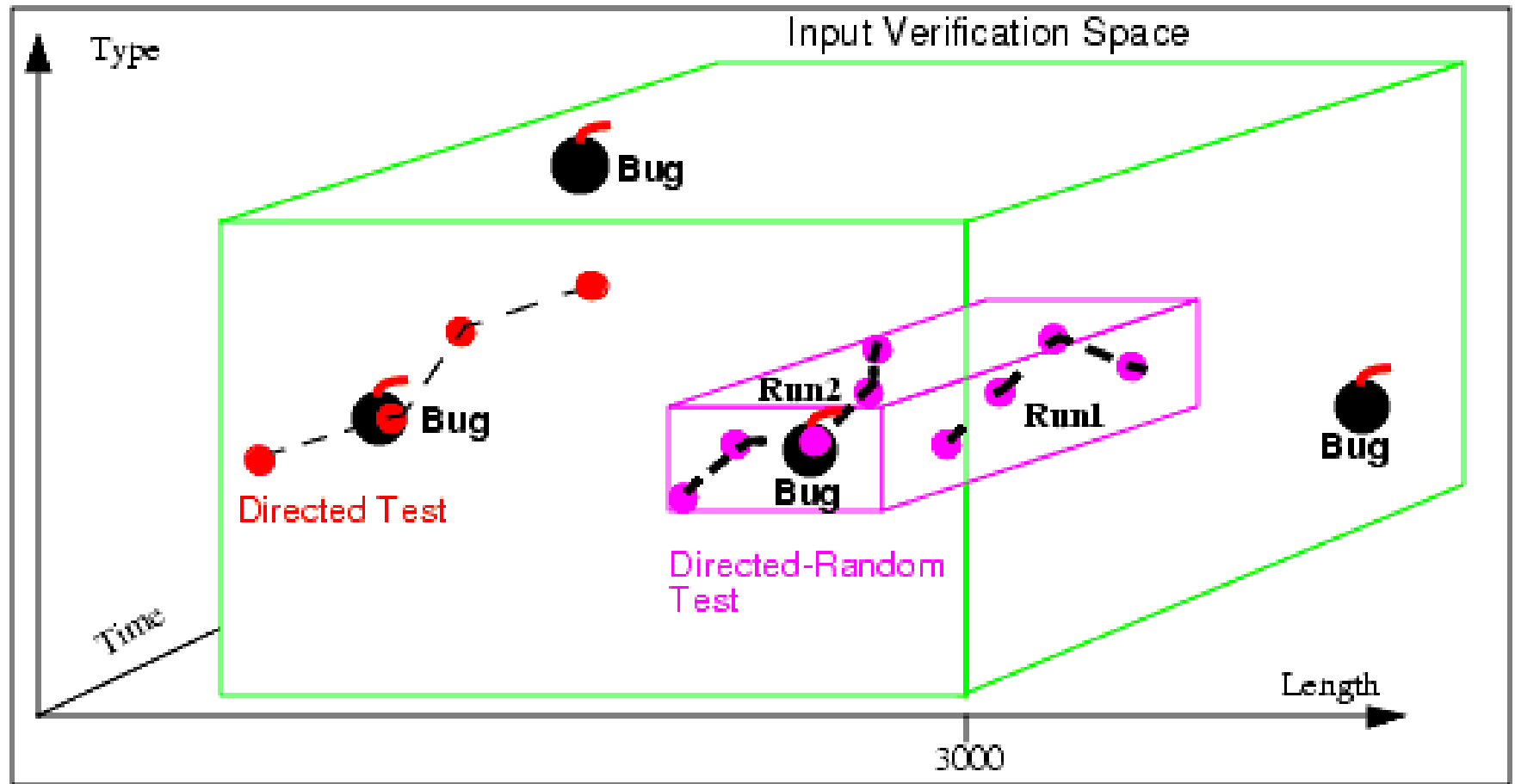
➤ פסבדו אקראיים

- החלטות אקראיות נשלטות על ידי גרעין (seed). בהינתן ערך של הגרעין: ההחלטות האקראיות הן דטרמיניסטיות
- **יצירה אקראית מוגבלת (Constrain Generation Problem)**
 - הגנרטור הגירויים מוגבל ליצור:
 - ❖ טסטים חוקיים
 - ❖ טסטים שעונים לדרישות המשתמש

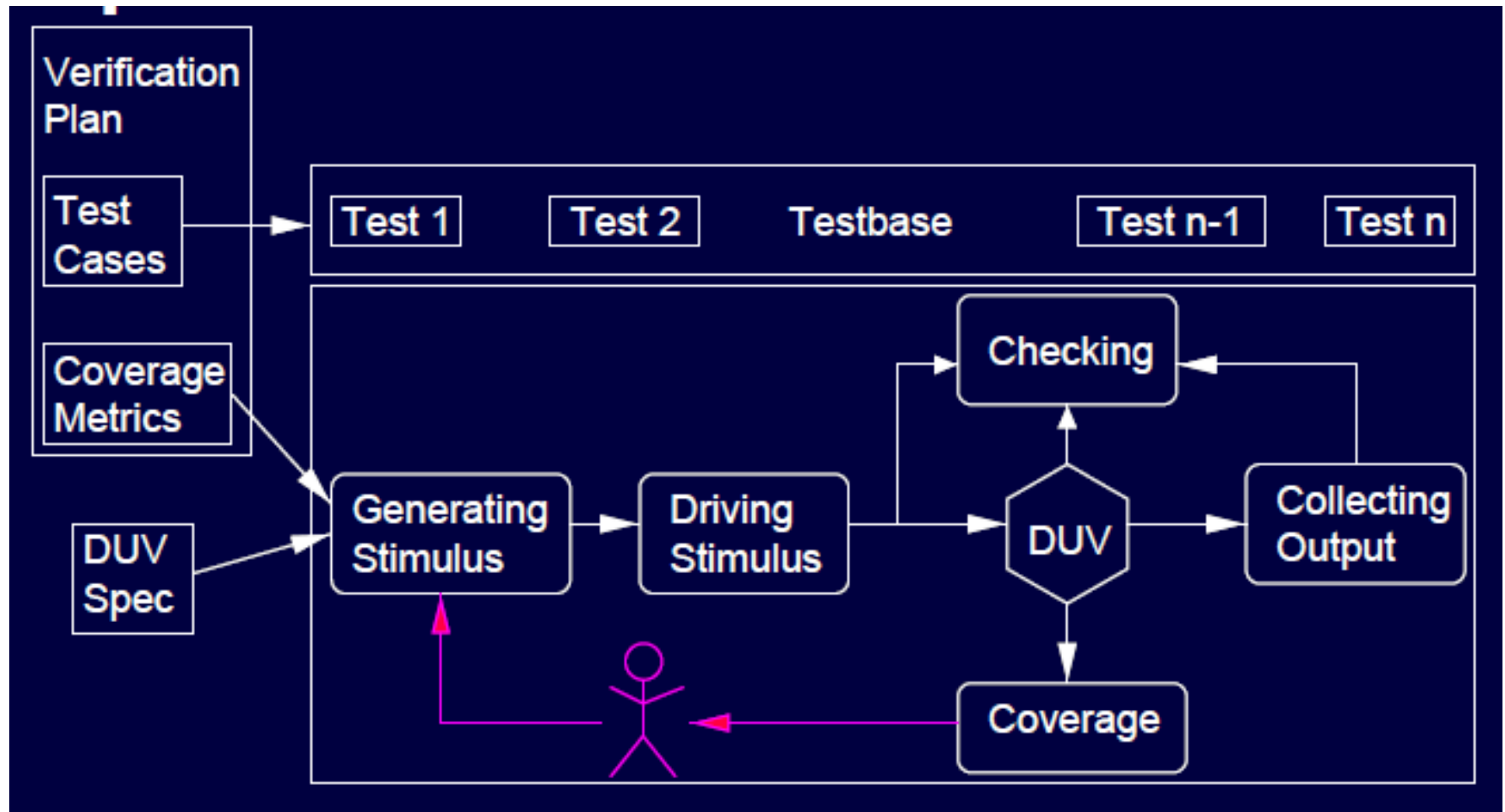
נשים את הכל בידך



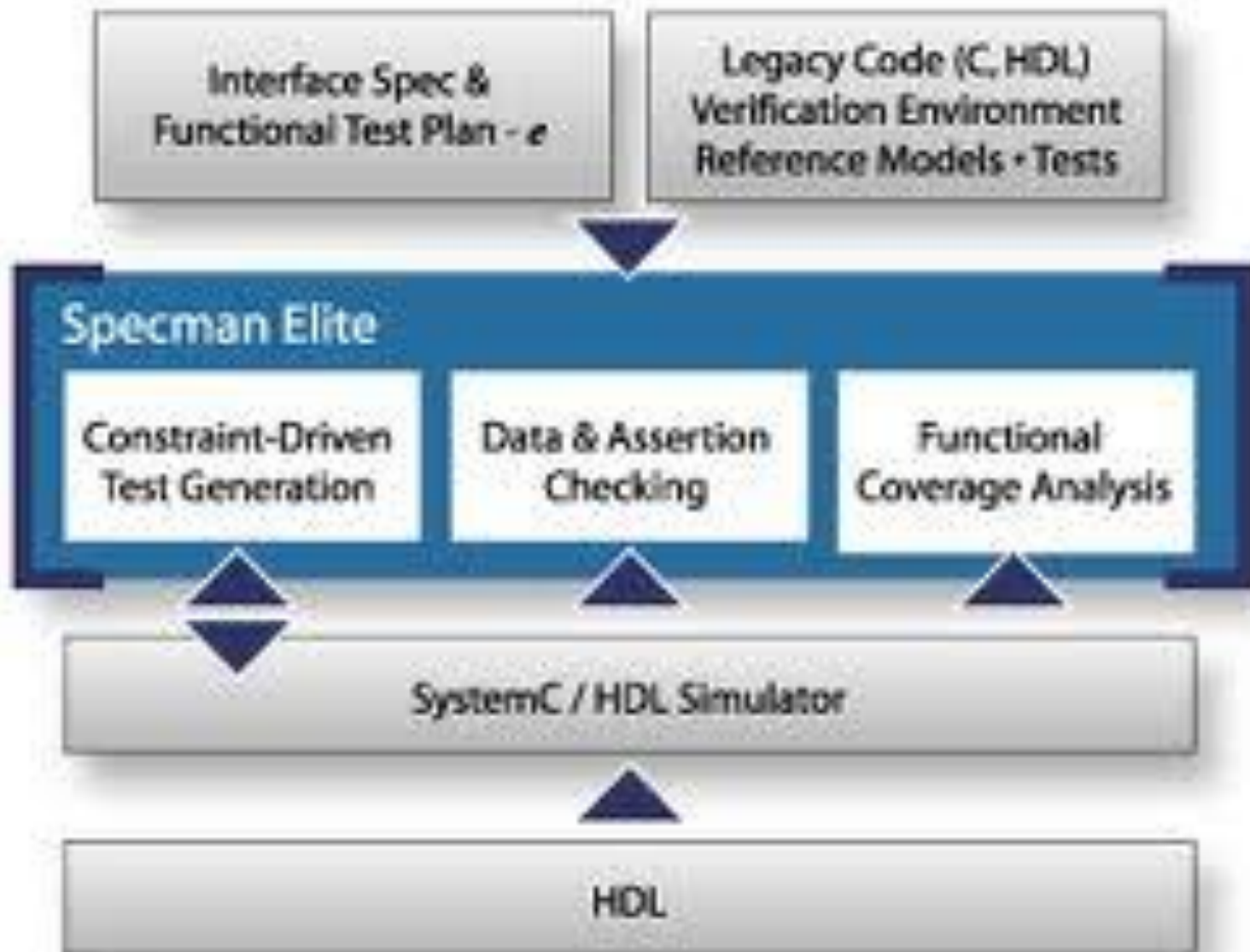
מרחב אקראי ומכוון



תהליך אימות מלא



SPECMAN



יסודות של שפה e

➤ שפה גבוהה לכתיבת סביבות האימות:

- הגדרת סביבת טסטים – testbenches
- טיפול בכיסוי
- גנרצית גירויים ובדיקות

➤ e תומך ב-:

- תכנון מודולרי aspect-oriented
- high-level data types

- גנרטור פסאודו אקראי מוגבל של נתונים מבוסס אירועים
- בדיקות ברמה גבוהה
- בדיקות המאפיינים הבסיסיים של תזמון



Aspect-Oriented Programming

➤ AOP הוא הצעד הבא למעלה מתכנות מונחה עצמים

- לטסטים יש מטרות ספציפיות:

- ❖ האם מספיק לבדוק זוגיות (parity)?

- ❖ האם תכונות התזמון של פרוטוקול העברת הנתונים תקפות?

➤ AOP מספק מנגנונים כדי להפריד בין שני חששות

(concerns) ליבטים נפרדים של סביבת אימות

- טכניקות מוגדרות היטב להוספת הצהרות, הוספת או החלפת הקוד מבחוץ של קלסים, ללא עריכת הקוד המקורי.

צורת התכנית

➤ קטע קוד e מוקף עם סימן להתחלת הקוד "<" וסימן סוף הקוד ">"

➤ כמה קטעי קוד יכולים להופיע בקובץ אחד, כל קטע כולל הצהרה אחת או יותר.

e code segment:

<

```
import cpu_test_env;
```

>

הערות -- Comments

➤ קבצי e מתחילים בהערה

➤ הערה זו מסתיימת כאשר נמצא "<

➤ ההערות בקטעי קוד יכולות להיות מסומנות ב- - או / /

רכיבים תחביריים -- Syntactic Elements

➤ הצהרות (Statements) נמצאות ברמה העליונה ביותר:

- ההצהרה תקפה בתוך סמנים ">" ו- "<"
- בסוף ההצהרה יש נקודה פסיק

➤ איברי struct נמצאים ברמה שנייה:

- תקפים אך ורק בתוך הגדרת struct
- הם קשורים עם מבנים דינמיים של testbench, זאת אומרת -- הגירויים.

רכיבים תחביריים

➤ הפעולות הנמצאות ברמה השלישית:

- תקפות רק כאשר הן קשורות לאיברי struct , כגון שיטה או אירוע.

➤ הביטויים הנמצאים ברמה הנמוכה ביותר:

- ניתן להשתמש אך ורק בתוך מבנה e אחר.

הצהרות -- Statements

הסוגים העיקריים של הצהרות:

- struct: הגדרת מבנה הנתונים החדש
(unit: מגדיר יחידה חדשה)
- type: מגדיר סוג ממוספר או תת-סוג
- extend: מרחיב struct או סוג שהוגדרו בעבר
- define: מרחיב את השפה
 - define OFFSET 5;
- import: ...

Structs vs Units

➤ המבנים הם אבני הבניין הבסיסיות ביותר של e:

- משמשים לשמור את הנתונים פעולים יחד

❖ packets, instructions, frames

- יכולים להיווצר בזמן הריצה, זאת אומרת הם דינמיים

Structs vs Units

➤ יחידות הם סוג מיוחד של מבנים

➤ יחידות הם סטטיות! יכולות להיווצר במהלך שלב הבדיקות בלבד

- מאפשרות מיפוי למסלול ה-HDL: הדרך הטובה ביותר להתחבר ל-DUV

- משמש עבור גנרטורים / בודקים (checkers) / מונטורים מודלים פונקציונליים של בוס (bus functional models) (BFMs), מבני בדיקה עצמית, testbench, ...

Struct and Struct Members

איברים נמצאים ברמה שניה: תקפים אך ורק בתוך הגדרת struct

➤ מבנה פשוט של חבילות לשימוש בפרוטוקול תקשורת:

```
type packet_kind: [atm, eth];  
struct packet {  
  len: int;  
  keep len < 256;  
  kind: packet_kind;  
};
```

אברי מבנה

- **Field:** Defines data entry to be member of enclosing struct with explicit data type.
- **Method:** Defines operational procedure that can manipulate fields of enclosing struct and access run-time values in DUV.
- **Subtype declaration:** Defines instance of parent struct in which specific members have particular values or behaviour.
 - Use when for conditional constraints on possible values of a field.

אברי מבנה

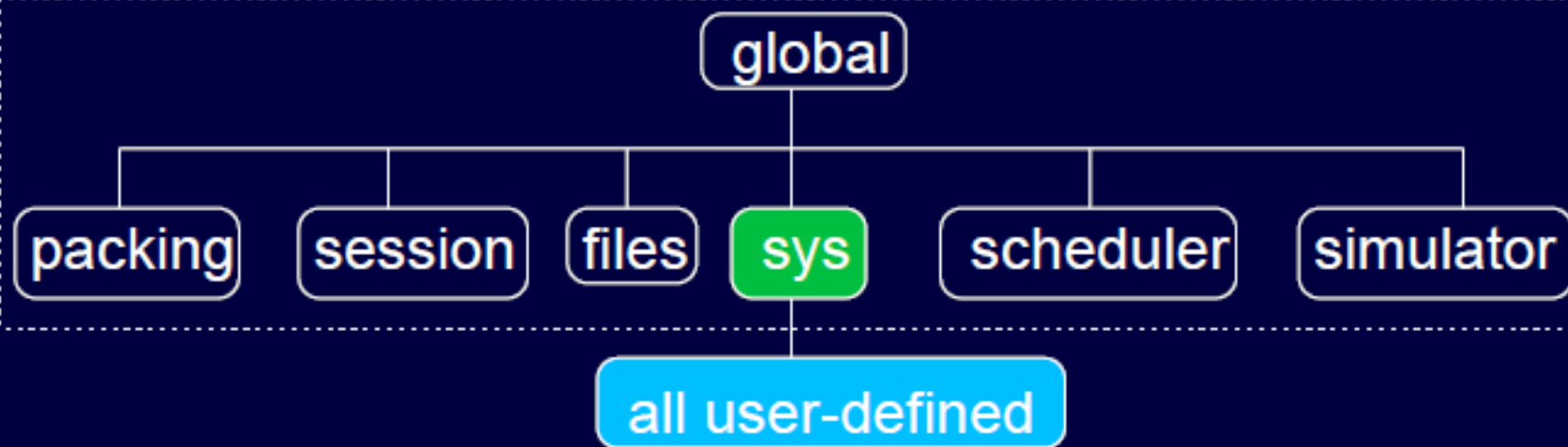
- **Constraint declaration:** Influences distribution of values generated for data entries and the order in which values are generated.
- **Coverage declaration:** Defines functional verification goals and collects data on how well the testbench is meeting these goals.
- **Temporal declaration:** Defines events and their associated actions.

סוגי נתונים המוגדרים מראש

Constant	Description
TRUE	For Boolean variables and expressions.
FALSE	For Boolean variables and expressions.
NULL	For structs, specifies a NULL pointer. For character strings, specifies an empty string.
UNDEF	UNDEF indicates NONE where an index is expected.
MAX_INT	Represents the largest 32-bit int ($2^{31} - 1$)
MIN_INT	Represents the smallest 32-bit int (-2^{31}).
MAX_UINT	Represents the largest 32-bit uint ($2^{32} - 1$).

המבנים המוגדרים מראש

בסביבת e מכילה מספר המבנים המוגדרים מראש (וכמובן כמה כאלה שמוגדרים על ידי המשתמש).



ישום תחת sys

➤ כל המבנים המוגדרים על ידי המשתמש כוללים יחידות מופעיות כמו (תת) שדה של sys

```
struct packet {  
    address : uint (bits : 2);  
    payload : uint (bytes : 64);  
};  
  
unit router_bfm {  
    packets : list of packet;  
};  
  
extend sys {  
    router : router_bfm is instance;  
};
```

גנרטור גירויים

➤ Offline לפני הסימולציה

- השתמש בפקודות generate או test
 - ❖ טסט קורא לפקודה generate
 - ❖ מייצר באופן רקורסיבי כל דבר תחת sys
 - ❖ היזהרו: יכול לצרוך הרבה זיכרון!

➤ Online בזמן הסימולציה

- השתמש פעולה gen
- מאפשר לייצר ערכים באופן דינאמי שמבוססים על המצב של DUV

שימוש באילוצים

keep constraint-bool-expr;

כאשר constraint-bool-expr הוא ביטוי בוליאני פשוט או מורכב

```
struct packet {  
  kind : [tx, rx];  
  len : int;  
  keep kind == tx => len==16;  
  --when tx packet { keep len == 16; }; exactly same effect  
};
```

Generation with keep

הסדר הגנרציה חשוב: זה משפיע על התפלגות הערכים!

```
struct packet {  
  kind : [tx, rx];  
  len : int;  
  keep len > 15 ==> kind==rx;  
};
```

1. אם `kind` נוצר הראשון, `kind` הוא שווה `tx` כמחצית הזמן
כי יש רק שני ערכים חוקיים של `kind`
2. אם `len` נוצר הראשון, ההתפלגות היא שונה

Fields

➤ שדות לא מיוצרים בצורה אוטומטית מתחילים מ-!

- זה שימושי עבור שדות שמוצבים באופן מפורש במהלך הבדיקה , או שערכים שלהם כרוכים בחישובים שלא ניתן לבטא בעזרת אילוצים

➤ שדות פיזיים שארוזים כאשר המבנה שלהם ארוז, מתחילים מ-%

- שדות שמייצגים נתונים שישלחו ל-HDL של הסימולטור או שימשו בזיכרונות הסימולטור או ב-Specman Elite צריכים להיות לשדות פיזיים

➤ לשדות אחרים נקראים לשדות וירטואליים לא ארוזים באופן אוטומטי כאשר המבנה שלהם ארוז, למרות שהם יכולים להיות ארוזים בצורה אינדיבידואלית

פעולות -- Actions

פעולות נמצאות ברמה שלישית ותקפים רק כאשר הם הקשורים לאיבר במבנה, כגון שיטה או אירוע

```
<'
struct packet{
    event xmit_ready is rise('top.ready');
    on xmit_ready {transmit();};
    transmit() is {
        out("transmitting packet...");
    };
};
'>
```



פעולות



➤ יצירת & שינוי משתנים

❖ var = , op=,

➤ אינטראקציה עם DUT

❖ force, release

➤ בקרת הזרימה

• משפטי תנאי

❖ if then else, labeled case, boolean case

• לולאות

❖ while, repeat until, for each, for from-to, for
each-line, for each-file-matching

שאלות על גנרציה והזרקה של גירויים

➤ מה שנוצר ומוזרק כל יציאה קלט?

➤ כמה יחידות מידע צריך להזריק לכל יציאה?

➤ איזה יחסים בין פריטי נתונים בזרם (ברצף) נדרשים?

➤ איזה קורלציות בין יציאות נדרשות?

שאלות על איסוף ובדיקות

- מה צריך להיות שנאסף?
- מה צריך להיות מוקלט?
- איך התוצאות הצפויות צריכות להיות מוכנות (מחושבות)?
- מה צריך להיבדק?
- איזה סוגים של נתונים יש להשוות?
- מה צריך לספור?
- האם בדיקות הזמנים נדרשות?
- האם צריך למדוד את הביצועים של DUT?

אימות מבוססת מפרט

